คู่มืออธิบายโครงงาน

Appliance Status Notifier

ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า

เอกสารสำหรับเตรียมนำเสนอโครงงาน
อธิบายตั้งแต่ระดับภาพรวมจนถึงรายละเอียดทางเทคนิค

1. ก่อนอื่น ระบบนี้คืออะไร?

ชื่อโครงงาน:

> **“ Appliance Status Notifier —** ระบบแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า

พูดง่ายที่สุดคือ:

> “ เราต้องการรู้ว่าอุปกรณ์ไฟฟ้ากำลังเปิดหรือปิดอยู่ โดยไม่จำเป็นต้องเดินไปดูที่ตัวอุปกรณ์

เช่น มีอุปกรณ์หนึ่งอยู่ไกลจากเรา

```
🔌 อุปกรณ์ไฟฟ้า
        │
        │ ?
        ▼
    เปิดอยู่ไหม?
```

ระบบของเราจะวัด กระแสไฟฟ้า ที่ไหลผ่านอุปกรณ์

ถ้ามีกระแสไหลในระดับที่เรากำหนด:

```
กระแสไหล
    ↓
น่าจะกำลังทำงาน
    ↓
🟢 เปิด
```

ถ้าแทบไม่มีกระแส:

```
ไม่มีกระแส
    ↓
น่าจะไม่ได้ทำงาน
    ↓
⚪ ปิด
```

จากนั้นส่งข้อมูลขึ้นอินเทอร์เน็ต

```
อุปกรณ์
  ↓
เซนเซอร์
  ↓
ESP32
  ↓
Internet
  ↓
```

```
Server
   ↓
Website
```

ผู้ใช้ก็เปิดเว็บดูได้

2. แล้วทำไมต้องวัดกระแส?

ตรงนี้เป็นหัวใจของโครงงาน

สมมติเรามีหลอดไฟ

```
🔋 ———— 💡
```

เมื่อเปิด:

```
🔋 ——→ กระแส ——→ 💡
```

เมื่อปิด:

```
🔋 ——————————————— 💡
          กระแสน้อยมาก
```

ดังนั้นเราไม่จำเป็นต้องติดกล้องเพื่อดูว่าหลอดไฟติดหรือไม่

เราใช้ พฤติกรรมทางไฟฟ้า เป็นตัวบอกสถานะ

นี่เป็นแนวคิดที่สำคัญมากในการนำเสนอ:

> “เราไม่ได้ตรวจสอบด้วยการมองเห็น แต่ตรวจสอบจากกระแสไฟฟ้าที่อุปกรณ์ใช้”

3. INA219 คืออะไร?

INA219 คือ เซนเซอร์วัดไฟฟ้า

มันสามารถวัดค่าที่เราสนใจ เช่น

- แรงดันไฟฟ้า (Voltage)
- กระแสไฟฟ้า (Current)
- กำลังไฟฟ้า (Power)

ตัวอย่าง:

```
Voltage = 5.01 V
Current = 320 mA
Power   = 1.60 W
```

คนทั่วไปไม่ต้องจำชื่อยาก ๆ ก็ได้

ให้จำว่า:

> **“ INA219 = เครื่องมือที่ทำให้ ESP32 รู้ว่าไฟกำลังไหลเท่าไร**

4. ทำไมมี INA219 สองตัว?

สมมติเราต้องการตรวจสอบอุปกรณ์ 2 ตัว:

```
🔌 อุปกรณ์ A
🔌 อุปกรณ์ B
```

เราต้องการ:

```
INA219 #1 → อุปกรณ์ A
INA219 #2 → อุปกรณ์ B
```

ดังนั้น:

```
          ESP32
         /     \
        /       \
  INA219 #1   INA219 #2
     |             |
     ▼             ▼
อุปกรณ์ A     อุปกรณ์ B
```

ทำให้เรารู้สถานะแต่ละตัวแยกกัน

5. แล้ว ESP32 ทำอะไร?

ให้คิดว่า ESP32 เป็น สมองของระบบ

INA219 เป็นคนที่บอกว่า:

> **“ “ตอนนี้มีกระแส 320 mA”**

ESP32 รับข้อมูลนั้นมาแล้วคิด:

> “ “320 mA มากกว่า **threshold** ที่กำหนด → อุปกรณ์กำลังทำงาน”

จึงได้:

```
INA219
   │
   │ 320 mA
   ▼
ESP32
   │
   │ มากกว่า threshold
   ▼
สถานะ = เปิด
```

ถ้า:

```
INA219
   │
   │ 0.5 mA
   ▼
ESP32
   │
   │ ต่ำกว่า threshold
   ▼
สถานะ = ปิด
```

6. แล้ว INA219 สองตัวคุยกับ ESP32 ยังไง?

ตรงนี้คือ **I²C**

ชื่ออาจฟังยาก แต่แนวคิดง่ายมาก

แทนที่จะลากสายแยกสำหรับ sensor แต่ละตัว เราสามารถให้หลายอุปกรณ์ใช้สายสื่อสารร่วมกันได้

ประมาณนี้:

```
ESP32
  │
  ├──── SDA ──────┬──── INA219 #1
  │               └──── INA219 #2
  │
  └──── SCL ──────┬──── INA219 #1
```

```
                    └── INA219 #2
```

SDA = สายส่งข้อมูล

SCL = สายจังหวะการสื่อสาร

7. แล้วทำไมสองตัวไม่สับสนกัน?

เพราะแต่ละ INA219 มี ที่อยู่ **(I²C Address)**

เหมือนบ้านเลขที่

```
INA219 #1 → บ้านเลขที่ 0x40
INA219 #2 → บ้านเลขที่ 0x41
```

ESP32 จึงสามารถบอกว่า:

> “ “ขอข้อมูลจาก **0x40**”

ได้ข้อมูลจากตัวแรก

หรือ:

> “ “ขอข้อมูลจาก **0x41**”

ได้ข้อมูลจากตัวที่สอง

นี่คือเหตุผลที่ sensor สองตัวสามารถอยู่บน bus เดียวกันได้

8. แล้ว ESP32 ส่งข้อมูลไปเว็บยังไง?

หลังจาก ESP32 อ่านข้อมูลแล้ว มันต้องส่งข้อมูลออกอินเทอร์เน็ต

ตัวอย่างข้อมูลแบบง่าย:

```
อุปกรณ์ A
Voltage = 5.01 V
Current = 320 mA
Power   = 1.60 W
Status  = ON
```

ESP32 เชื่อม Wi-Fi แล้วส่งข้อมูลไป Server

```
ESP32
```

```
   |
   | Wi-Fi
   ▼
Internet
   |
   ▼
Cloudflare Worker
```

9. Cloudflare Worker คืออะไร?

อย่าอธิบายว่าเป็น serverless edge runtime ตั้งแต่แรก 😂

พูดง่าย ๆ:

> **“ Cloudflare Worker** เป็นตัวกลางที่รับข้อมูลจาก **ESP32** และให้เว็บไซต์เข้ามาขอข้อมูล

เหมือนพนักงานกลาง:

```
ESP32
   |
   | "นี่คือข้อมูลล่าสุด"
   ▼
┌──────────────┐
│    Worker    │
└──────────────┘
   ▲
   |
   | "ขอสถานะล่าสุด"
   |
Website
```

ข้อดีคือ ESP32 ไม่จำเป็นต้องคุยกับหน้าเว็บไซต์โดยตรง

10. ทำไมไม่ให้เว็บคุยกับ ESP32 โดยตรง?

เพราะ ESP32 อาจอยู่ในบ้าน/ห้อง/พื้นที่อื่น

และไม่ได้มี public server ให้เว็บเข้าถึงโดยตรง

จึงให้:

```
ESP32 → Internet → Server ← Internet ← Website
```

แทน

ทำให้ระบบแยกหน้าที่กัน:

```
ESP32
= เก็บข้อมูล

Worker
= จัดการข้อมูล

Website
= แสดงข้อมูล
```

11. แล้วข้อมูลเก็บไว้ที่ไหน?

ระบบของคุณมีสองส่วนหลัก:

KV

คิดง่าย ๆ ว่า:

> “กระดาษแผ่นล่าสุด”

เก็บว่า:

```
ตอนนี้สถานะเป็นอะไร?
```

เช่น:

```
ESP32-01
ล่าสุด:
INA219 #1 = เปิด
INA219 #2 = ปิด
```

เหมาะกับการอ่านสถานะปัจจุบันอย่างรวดเร็ว

12. D1 คืออะไร?

D1 เป็นฐานข้อมูล

ให้คิดว่าเป็น:

> “สมุดบันทึกประวัติ”

เช่น:

```
10:00 → เปิด
10:15 → เปิด
10:30 → ปิด
10:45 → เปิด
```

ดังนั้น:

```
KV
↓
สถานะล่าสุด

D1
↓
ประวัติย้อนหลัง
```

นี่เป็นประโยคที่เอาไปนำเสนอได้เลย:

> “เราแยกการเก็บข้อมูลปัจจุบันออกจากข้อมูลประวัติ เพื่อให้การอ่านสถานะปัจจุบันทำได้รวดเร็ว และยังสามารถเก็บข้อมูลย้อนหลังไว้สำหรับวิเคราะห์ได้”

ถ้ากรรมการถามลึก นี่ตอบได้ดีมาก

13. แล้ว Website ทำอะไร?

เว็บไซต์ไม่ได้ไปอ่าน INA219 โดยตรง

มันถาม Worker:

```
Website
   │
   │ GET /status
   ▼
Worker
   │
   ▼
KV
   │
   ▼
```

```
ข้อมูลล่าสุด
    |
    ▼
Website
```

แล้วเอาข้อมูลมาแสดง:

```
┌──────────────────────────────┐
| Appliance Status             |
|                              |
| อุปกรณ์ 1                    |
| 🟢 เปิด                      |
| 5.01 V                       |
| 320 mA                       |
| 1.60 W                       |
|                              |
| อุปกรณ์ 2                    |
| ⚪ ปิด                       |
| 5.02 V                       |
| 0.8 mA                       |
| 0.004 W                      |
└──────────────────────────────┘
```

14. แล้วถ้า ESP32 หายไปล่ะ?

อันนี้สำคัญมากสำหรับ ความเสถียร

สมมติ ESP32 ส่งข้อมูลทุก 10–15 วินาที

แต่เว็บไซต์ไม่ได้รับข้อมูลใหม่เลย:

```
ESP32
  ↓
ส่งข้อมูล
  ↓
10 sec
  ↓
ส่งข้อมูล
  ↓
15 sec
  ↓
❌ หาย
```

เว็บไซต์ไม่ควรบอกว่า:

> 🟢 เปิด

เพราะนั่นเป็น ข้อมูลเก่า

ระบบจึงกำหนด timeout เช่น:

```
ข้อมูลใหม่ภายใน 30 วินาที
          ↓
       Online

เกิน 30 วินาที
          ↓
        N/A
```

ดังนั้น:

```
🟢 เปิด
```

หมายถึง:

> "ข้อมูลล่าสุดยังใหม่พอ"

ไม่ใช่:

> "เรารับประกันว่าอุปกรณ์เปิดอยู่แน่นอน"

อันนี้เป็น distinction ที่ดีมากในเชิง engineering

15. แล้ว LINE ทำอะไร?

ระบบสามารถสรุปข้อมูลประจำวันแล้วส่งผ่าน LINE Official Account

Flow:

```
ข้อมูลตลอดวัน
      ↓
     D1
      ↓
16:10
      ↓
Cloudflare Cron
      ↓
สรุปข้อมูล
      ↓
```

```
LINE Messaging API
        ↓
📱 ผู้ใช้
```

ตัวอย่าง:

```
📊 สรุปสถานะประจำวัน

อุปกรณ์ 1
🟢 เปิด 5 ชั่วโมง
⚪ ปิด 3 ชั่วโมง

อุปกรณ์ 2
🟢 เปิด 2 ชั่วโมง
⚪ ปิด 6 ชั่วโมง
```

ดังนั้น Website กับ LINE มีหน้าที่ต่างกัน:

```
Website
→ ดูสถานะปัจจุบัน

LINE
→ รับสรุป/แจ้งเตือน
```

16. แล้ว API คืออะไร?

ถ้ากรรมการถามว่า:

> “API คืออะไร?”

ตอบง่าย ๆ:

> “API คือช่องทางที่กำหนดไว้ให้โปรแกรมต่าง ๆ สามารถส่งหรือขอข้อมูลจากกันได้”

เช่น:

```
POST /update
```

หมายถึง:

> “ESP32 ส่งข้อมูลใหม่เข้ามา”

ส่วน:

```
GET /status
```

หมายถึง:

> “เว็บไซต์ขอสถานะล่าสุด”

และ:

```
GET /health
```

หมายถึง:

> “ระบบยังทำงานอยู่หรือไม่?”

17. JSON คืออะไร?

ไม่ต้องกลัว

มันคือ รูปแบบการจัดข้อมูล

แทนที่จะส่งข้อมูลมั่ว ๆ:

```
ESP32-01 0x40 5.01 320 1600 true
```

เราส่งแบบมีความหมาย:

```
{
  "device_id": "ESP32-01",
  "sensors": [
    {
      "sensor_id": "INA219-01",
      "voltage_v": 5.01,
      "current_ma": 320,
      "power_mw": 1600,
      "is_on": true
    }
  ]
}
```

คนทั่วไปไม่จำ JSON ทั้งหมด

จำแค่ว่า:

> “เป็นรูปแบบมาตรฐานสำหรับจัดข้อมูลให้โปรแกรมแต่ละส่วนเข้าใจตรงกัน”

18. แล้วทำไมต้องมี device_id และ sensor_id?

เพราะ:

```
ESP32-01
  ├── INA219-01
  └── INA219-02
```

สองอย่างนี้คนละความหมาย

Device

คือเครื่อง/บอร์ด:

```
ESP32-01
```

Sensor

คืออุปกรณ์ตรวจวัด:

```
INA219-01
INA219-02
```

ดังนั้นถ้าวันหนึ่งมี:

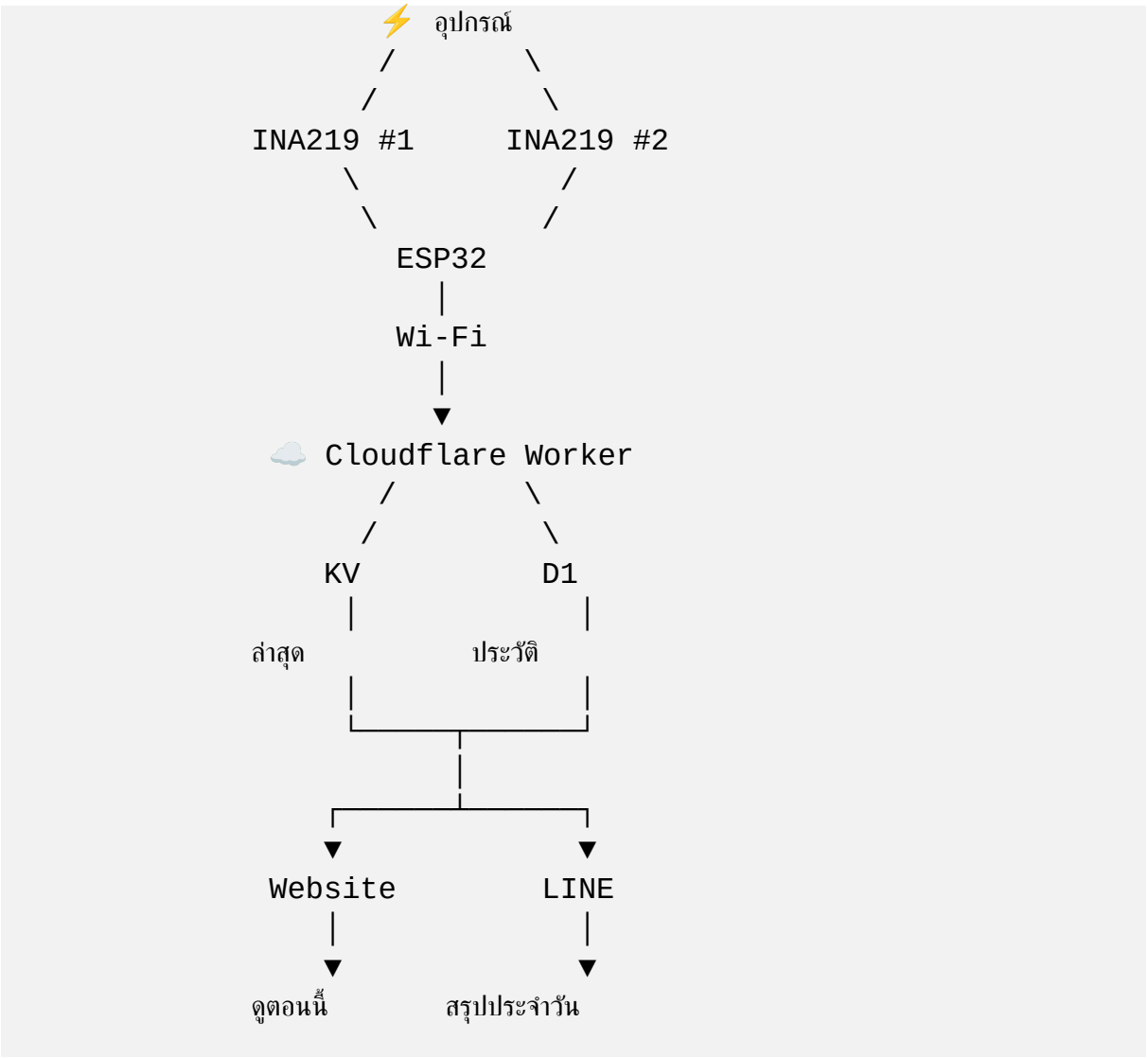
```
ESP32-01
  ├── INA219-01
  └── INA219-02

ESP32-02
  ├── INA219-01
  └── INA219-02
```

ก็ยังแยกข้อมูลได้

19. แล้วระบบทั้งหมดรวมกันเป็นอะไร?

นี่คือภาพที่ผมแนะนำให้เอาไปใช้ในการนำเสนอ:



ถ้าคนฟังเข้าใจรูปนี้ได้ = เข้าใจระบบเกือบทั้งหมดแล้ว

20. แล้วเรื่องความเสถียรล่ะ?

นี่คือส่วนที่ทำให้โครงงานคุณน่าสนใจขึ้น

ระบบจริงไม่ได้คิดแค่:

> "ทำให้มันทำงาน"

แต่ต้องคิด:

> "ถ้ามันไม่ทำงาน จะเกิดอะไรขึ้น?"

ตัวอย่าง:

Sensor เสีย

```
INA219 #1 ❌
INA219 #2 ✅
```

เราไม่ควรให้ทั้งระบบตาย

ควรระบุ:

```
Sensor #1 = Error
Sensor #2 = Normal
```

Wi-Fi หลุด

```
ESP32
  ↓
Wi-Fi ❌
```

ESP32 ควรพยายาม reconnect

และไม่ควรทำให้ระบบค้างถาวร

ESP32 หาย

```
ESP32 ❌
      ↓
ไม่มีข้อมูลใหม่
      ↓
Website
      ↓
N/A
```

Backend รับข้อมูลผิด

```
ESP32
  ↓
ข้อมูลผิด
  ↓
Worker
  ↓
ตรวจสอบ
```

```
  ↓
Reject
```

ไม่ควรเอาข้อมูลที่ผิดเข้า database

21. แล้ว STEM อยู่ตรงไหน?

ตรงนี้สำคัญถ้าขึ้นไปเจอกรรมการจริงจัง

โครงงานนี้ไม่ได้มีแค่ programming

มันมี:

Science

ศึกษาพฤติกรรมของกระแสไฟฟ้าและการใช้พลังงาน

```
Voltage
Current
Power
```

Mathematics

ใช้ threshold, การคำนวณ และการวิเคราะห์ข้อมูล

เช่น:

[

P = VI

]

กำลังไฟฟ้า = แรงดัน × กระแส

Engineering

ออกแบบระบบให้:

- sensor ทำงาน
- communication ทำงาน
- failure แล้วไม่พังทั้งระบบ

- มีการทดสอบ
- มีข้อจำกัดและการป้องกัน

Technology

ใช้:

```
ESP32
I²C
Wi-Fi
HTTP
Cloudflare
Database
Web
LINE API
```

ดังนั้นมันเป็นโครงงานที่แตะ **STEM** ครบค่อนข้างชัด

22. ถ้าต้องอธิบายทั้งโครงงานใน 2 นาที

ผมจะให้คนในกลุ่มพูดประมาณนี้:

> “ “โครงงานของเราคือระบบตรวจสอบและแจ้งเตือนสถานะอุปกรณ์ไฟฟ้า โดยแนวคิดหลักคือการตรวจสอบสถานะของอุปกรณ์จากกระแสไฟฟ้าที่ไหลผ่าน”
> “
>
> “ “เราใช้ **INA219** เป็นเซนเซอร์สำหรับวัดแรงดันและกระแส และใช้ **ESP32** เป็นตัวประมวลผล โดยในระบบสามารถใช้ **INA219** สองตัวเพื่อแยกตรวจสอบอุปกรณ์แต่ละตัวได้”
> “
>
> “ “**ESP32** จะอ่านค่าจาก **INA219** ผ่านการสื่อสารแบบ **I²C** จากนั้นนำค่ากระแสมาเปรียบเทียบกับค่าที่กำหนดไว้ เพื่อประเมินว่าอุปกรณ์กำลังทำงานหรือไม่”
> “
>
> “ “เมื่อได้ข้อมูลแล้ว **ESP32** จะส่งข้อมูลผ่าน **Wi-Fi** ไปยัง **Cloudflare Worker** ซึ่งทำหน้าที่เป็นตัวกลางระหว่าง **ESP32** กับเว็บไซต์ โดยข้อมูลสถานะล่าสุดจะเก็บไว้ใน **KV** และข้อมูลประวัติจะเก็บไว้ใน **D1**”
> “
>
> “ “เว็บไซต์จะขอสถานะล่าสุดจากระบบกลางและนำมาแสดงให้ผู้ใช้เห็น และหากไม่มีข้อมูลใหม่เกินเวลาที่กำหนด เว็บไซต์จะแสดง **N/A** เพื่อป้องกันการแสดงข้อมูลเก่าราวกับว่าเป็นข้อมูลปัจจุบัน”

> “

> “ “นอกจากนี้ระบบยังสามารถใช้ข้อมูลประวัติเพื่อสร้างสรุปและส่งแจ้งเตือนผ่าน **LINE Official Account** ตามเวลาที่กำหนด”

> “

> “ “ดังนั้นระบบของเราจึงประกอบด้วยส่วนวัดข้อมูล ส่วนประมวลผล ส่วนสื่อสาร ส่วนจัดเก็บข้อมูล และส่วนแสดงผล ซึ่งแต่ละส่วนมีหน้าที่แยกกันอย่างชัดเจน”

นี่คือ **version** ที่คนทั่วไปฟังแล้วตามทัน แต่ถ้ากรรมการอยากถามลึก คุณก็สามารถเปิดแต่ละ layer ต่อได้

และผมคิดว่าแก่นที่ทุกคนในกลุ่มควรจำมีแค่ **7** ประโยค

ถ้าทุกคนจำ 7 อันนี้ได้ ผมถือว่า พร้อมนำเสนอระดับพื้นฐานแล้ว:

> “ **1. INA219** ทำหน้าที่วัดไฟฟ้า

> “ **2. ESP32** ทำหน้าที่อ่านข้อมูลและตัดสินสถานะ

> “ **3. I²C** เป็นวิธีที่ **ESP32** ใช้สื่อสารกับ **INA219**

> “ **4. Wi-Fi** ใช้ส่งข้อมูลจาก **ESP32** ไปยังระบบกลาง

> “ **5. Cloudflare Worker** เป็นตัวกลางจัดการข้อมูล

> “ **6. KV** เก็บสถานะล่าสุด ส่วน **D1** เก็บประวัติ

> “ **7. Website** แสดงผลปัจจุบัน ส่วน **LINE** ใช้สำหรับการแจ้งเตือน

จากนั้นค่อยมี **technical layer** สำหรับคุณ/คนที่ทำแต่ละส่วน เช่น `0x40/0x41`, I²C addressing, JSON schema, HTTP, retry, timeout, database schema, current threshold, sampling interval, EMI/noise, power supply และ failure handling

แบบนี้จะไม่กลายเป็นว่า “คนทั้ง **6** คนต้องเข้าใจ **Cloudflare Worker** ระดับเดียวกับคนเขียน **backend**” แต่ทุกคนยังสามารถอธิบายระบบเดียวกันจากภาพรวมเดียวกันได้ครับ.